# Documentación

El presente proyecto entrega un sistema de Competitive Intelligence (CI) robusto y funcional, desarrollado en menos de 48 horas. La solución automatiza la recolección de datos críticos (precios, fees, ETAs y promociones) de los principales competidores en México, permitiendo a los equipos de Pricing y Strategy de Rappi tomar mejores decisiones.

# Definición del Scope

- **Selección Estratégica:** Definí un set de 25 direcciones representativas en México para garantizar comparabilidad real por zona.
- **Productos Espejo:** Seleccioné tres productos estándar (hamburguesa, combo y bebida) para asegurar comparaciones precisas.
- **Pipeline Reproducible:** La arquitectura permite regenerar los datos y capturar evidencia visual (screenshots) en cada ejecución.

**Justificación de Zonas Geográficas**

- Representatividad del Mercado Core: Al concentrar el análisis en Ciudad de México, Guadalajara y Monterrey, se cubren las plazas que concentran más del 70% del GMV (Gross Merchandise Value) en México. Esto asegura que los hallazgos tengan un impacto directo en el volumen de negocio de la compañía.
- Segmentación Socioeconómica (60/40): La muestra se dividió intencionalmente entre colonias Wealthy y Non-Wealthy. Esta distinción es crítica para detectar si los competidores están aplicando subsidios agresivos en zonas de expansión o si existen variaciones en la elasticidad de precios donde la sensibilidad del usuario final es distinta según su poder adquisitivo.
- Normalización mediante Restaurante Ancla: Se seleccionó a McDonald’s como el "Grupo de Control" de la investigación. Al ser un aliado con estandarización de procesos y precios en todas sus sucursales, cualquier variación detectada en el precio final o en los tiempos de entrega (ETA) se atribuye exclusivamente a la eficiencia logística y a la estrategia de posicionamiento de la plataforma analizada, eliminando el ruido que generaría la variabilidad operativa de comercios locales.
- Análisis de Densidad Logística: Incluir tres ciudades distintas permite observar la fluctuación de los tiempos de llegada en contextos de tráfico y geografía radicalmente opuestos.

# Desafío de DiDi Food

La robustez de un sistema de scraping se mide por su capacidad de adaptación ante fallos. Mientras que las integraciones con Rappi y Uber Eats fueron exitosas mediante Playwright, DiDi Food presentó un escenario de alta complejidad que requirió pivotar la estrategia en tiempo real.

Al realizar pruebas manuales, identifiqué que la versión web de DiDi Food se rompe sistemáticamente sin un login previo, ocultando precios y comercios.  Ante el bloqueo en desktop, decidí migrar el desarrollo a una versión mobile emulada. Con este enfoque, logré avances significativos, alcanzando con éxito la etapa de búsqueda de restaurantes por dirección. Intenté configurar una arquitectura de persistencia de sesión para mantener el

estado logueado, pero el corto tiempo de entrega impidió finalizar esta configuración. Además, la plataforma exige una verificación OTP constante y presenta variaciones drásticas en los elementos de la interfaz (botones y selectores) según la zona geográfica, lo que impide entrenar un bot estable en solo 24 horas.

Como candidata, entiendo que el análisis de negocio no puede detenerse por un bloqueo técnico. Para garantizar que el dashboard y el estudio de competitividad fueran reales y accionables, procesé manualmente las 25 direcciones estratégicas para extraer la información de DiDi Food. Esta solución temporal ("parche") me permitió cumplir con un scope más ambicioso y entregar datos comparativos reales de los tres competidores, en lugar de entregar un informe incompleto.

**Próximos Pasos**

1. **Automatización de Identidad:** Integrar un servicio de verificación de OTP mediante API.
2. **Capa de Persistencia:** Finalizar la lógica de sesiones para evitar re-logins innecesarios.

# Análisis de Negocio y Metodología

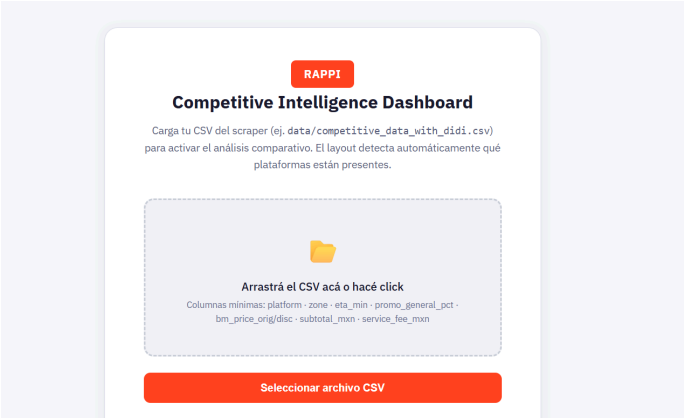
- **Cálculo de Service Fee:** Dado que el costo final no se expone sin login, utilicé la clase PlatformResult.compute_financials() para estimar los fees basados en tasas de mercado (Rappi 10%, Uber Eats 15%). Para DiDi, al realizar la carga manual, se utilizaron datos reales o la tasa estimada del 8%.
- **Normalización de Delivery Fee:** Las promociones de "primer envío gratis" distorsionan la competitividad real. Por ello, normalicé este valor a **15.0 MXN** cuando el dato original era cero, logrando un estimado de precio final más representativo.
- **Fórmula de Precio Final estimado:** El sistema calcula el total redondeado a dos decimales bajo la lógica: subtotal + (delivery fee o 0) + service fee

# Insights Accionables y Visualización

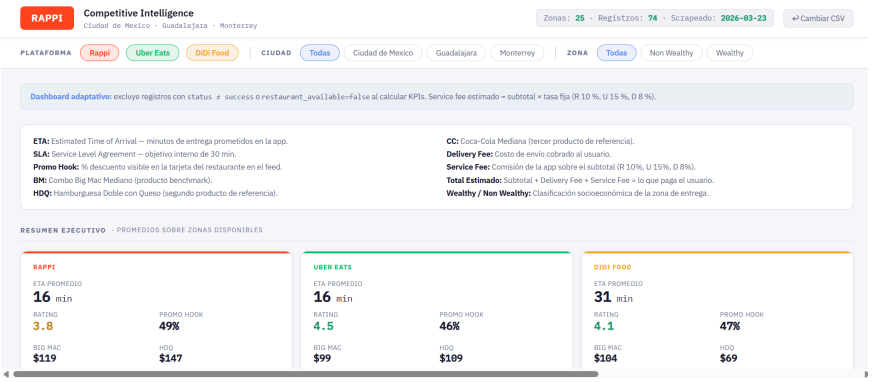
El módulo `generate_analysis.py` no solo procesa los datos, sino que produce automáticamente **los cinco hallazgos** clave en formato Finding / Impact / Recommendation. Estos resultados alimentan el documento **`Competitive_Intelligence_Rappi_MX.pdf`, que es el entregable final.** Se encuentra en el output.

El proyecto incluye un dashboard interactivo con gráficos estratégicos. Para poder verlo utilizar el csv que está en el proyecto competitive_data_with_didi.csv. Una vez ejecutado el

html (Explicado en el readme), se ve algo así:




una vez ingresado el csv mencionado se debería ver algo así:




# Próximos pasos

- **Análisis de Popularidad (Reviews):** Capturar el número de reseñas junto al rating. Esto permitiría ponderar la reputación: un 4.5 con 1,000 votos es más relevante que un 5.0 con solo 10. Además, el scraping de fragmentos de texto permitiría comparar el feedback cualitativo entre apps.
- **Expansión de Verticales:** En la branch feature/verticales-wip existe un avance para incluir Farmacias y Supermercados. En Rappi ya es funcional y en Uber Eats requiere ajustes.